

ENGENHARIA DA QUALIDADE E CONFIABILIDADE

Métricas Google SRE

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



- 1 Introdução às métricas
- 2 Downtime Não Planejado
- 3 Disponibilidade do Serviço
- 4 Disponibilidade em sistemas distribuídos
- 5 O que é uma API REST?
- 6 O que é uma requisição?
- 7 Métodos HTTP
 - GET
 - POST
 - PUT
 - DELETE
- 8 Disponibilidade Baseada em Requisições

Prática-Padrão

Como prática-padrão no Google, busca-se definir uma **métrica objetiva** para representar a propriedade que queremos otimizar.

Por que usar métricas?

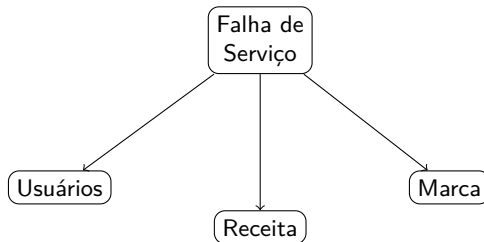
- Avaliar o desempenho atual
- Monitorar melhorias
- Identificar degradações ao longo do tempo

Desafio

Nem sempre é simples reduzir todos os riscos a uma única métrica.

Consequências Potenciais

- Insatisfação dos usuários
- Perda de confiança
- Perda de receita
- Impacto na marca
- Cobertura negativa da imprensa



Foco no Downtime Não Planejado

Por que focar no Downtime?

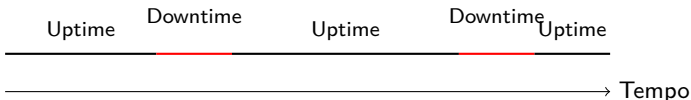
Ao lidar com riscos em sistemas complexos, existem muitos impactos possíveis (financeiros, reputacionais, operacionais). Para garantir **padronização** e comparabilidade entre diferentes serviços, escolhe-se uma métrica comum: **o downtime não planejado**.

Definição Formal

Downtime não planejado corresponde aos períodos em que o serviço fica indisponível sem que essa interrupção tenha sido previamente programada ou comunicada.

Importância

- Afeta diretamente a experiência do usuário
- Impacta SLAs e contratos
- Pode gerar perdas financeiras
- Compromete a confiança no sistema



Disponibilidade do Serviço

Conceito de Disponibilidade

Disponibilidade representa a probabilidade de um sistema estar operacional e acessível quando requisitado pelo usuário. Em termos práticos, expressa o nível tolerável de downtime não planejado.

Expressão em "Noves"

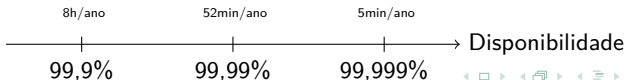
A disponibilidade é tradicionalmente representada pelo número de "noves":

- 99,9% → até 8h45min de indisponibilidade por ano
- 99,99% → até 52min por ano
- 99,999% → cerca de 5min por ano

Cada nove adicional reduz o downtime em aproximadamente **uma ordem de magnitude**.

Interpretação

Pequenas diferenças percentuais representam grandes diferenças no tempo total de indisponibilidade anual.

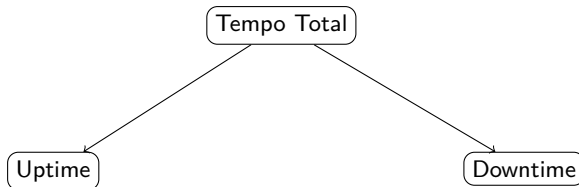


Equação 3.1

$$\text{Disponibilidade} = \frac{\text{Tempo de Uptime}}{\text{Tempo Total}}$$

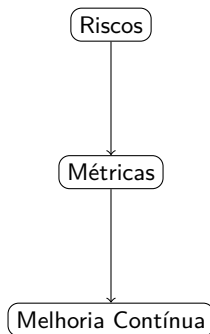
Interpretação

- Mede proporção de tempo operacional
- Base da engenharia de confiabilidade



Principais Pontos

- Riscos devem ser traduzidos em métricas
- Foco no downtime não planejado
- Disponibilidade expressa em “noves”
- Métricas permitem melhoria contínua



Por que Métricas Geram Melhoria Contínua?

1. Tornam o desempenho observável

O que não é medido permanece no campo da percepção. Métricas transformam impressões subjetivas em dados objetivos, permitindo análise técnica e comparação temporal.

2. Criam ciclos de feedback

A melhoria contínua depende de realimentação. Ao medir desempenho antes e depois de uma intervenção, é possível avaliar impacto, validar hipóteses e corrigir estratégias.

3. Permitem definição de metas claras

Indicadores quantitativos viabilizam:

- Estabelecimento de SLAs
- Comparação com benchmarks
- Priorização de investimentos

Sem métricas, decisões tendem a ser reativas.



Ciclo Estruturado de Aprendizado

A medição sistemática viabiliza ciclos iterativos como o PDCA:

- **Plan** – definir metas (ex: reduzir MTTR)
- **Do** – implementar melhorias
- **Check** – medir resultados
- **Act** – ajustar processos

Sem a etapa de medição, o ciclo se interrompe.

Efeitos Organizacionais

Métricas adequadas:

- Alinham incentivos e prioridades
- Reduzem ambiguidade na tomada de decisão
- Permitem identificar tendências (melhoria ou degradação)
- Transformam incidentes em aprendizado cumulativo

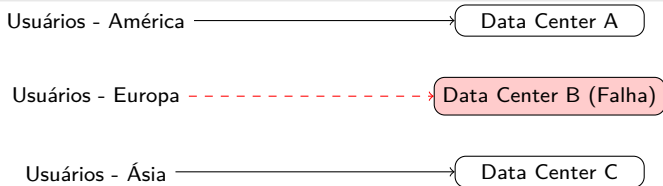
Problema da Disponibilidade Baseada em Tempo em Sistemas Distribuídos

Contexto em Sistemas Distribuídos Globais

- Múltiplos data centers geograficamente distribuídos
- Isolamento regional de falhas
- Impacto parcial de tráfego
- Sistema quase nunca 100% down

Implicação

Medir disponibilidade apenas como “tempo totalmente fora do ar” torna-se pouco representativo, pois o sistema pode estar parcialmente operacional durante toda a falha.



Sistema parcialmente operacional

O que é uma API REST?

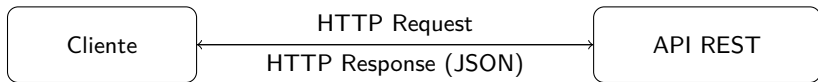
Definição

API (Application Programming Interface) é um conjunto de regras que permite que sistemas se comuniquem entre si.

REST (Representational State Transfer) é um estilo arquitetural baseado em recursos acessados via HTTP.

Características Principais

- Comunicação cliente-servidor
- Uso de métodos HTTP (GET, POST, PUT, DELETE)
- Recursos identificados por URLs
- Respostas normalmente em JSON



O que é uma Requisição?

Definição

Uma requisição é uma mensagem enviada por um cliente a um servidor solicitando a execução de uma operação sobre um recurso.

Componentes de uma Requisição HTTP

- Método (GET, POST, PUT, DELETE)
- URL (recurso desejado)
- Cabeçalhos (headers)
- Corpo (body) — opcional

```
POST /usuarios  
Host: exemplo.com  
Content-Type: application/json  
{ "nome": "Ana" }
```

Método HTTP: GET

O que é?

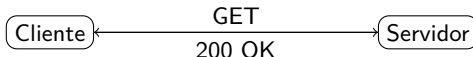
Obtém dados de um recurso no servidor. Não altera estado. Seguro e idempotente.

Exemplo de Requisição

```
GET /usuarios/10 HTTP/1.1
Host: api.exemplo.com
Accept: application/json
```

Exemplo de Resposta

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "id": 10,
  "nome": "Ana",
  "email": "ana@email.com"
}
```



Método HTTP: POST

O que é?

Cria um novo recurso no servidor. Não é idempotente.

Exemplo de Requisição

```
POST /usuarios HTTP/1.1
Host: api.exemplo.com
Content-Type: application/json
{
  "nome": "Carlos",
  "email": "carlos@email.com"
}
```

Exemplo de Resposta

```
HTTP/1.1 201 Created
Location: /usuarios/11
```



Método HTTP: PUT

O que é?

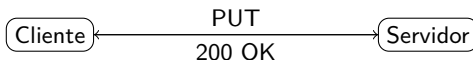
Atualiza completamente um recurso existente. É idempotente.

Exemplo de Requisição

```
PUT /usuarios/10 HTTP/1.1
Host: api.exemplo.com
Content-Type: application/json
{
  "nome": "Ana Souza",
  "email": "ana@email.com"
}
```

Exemplo de Resposta

HTTP/1.1 200 OK



Método HTTP: DELETE

O que é?

Remove um recurso do servidor. É idempotente.

Exemplo de Requisição

```
DELETE /usuarios/10 HTTP/1.1  
Host: api.exemplo.com
```

Exemplo de Resposta

```
HTTP/1.1 204 No Content
```

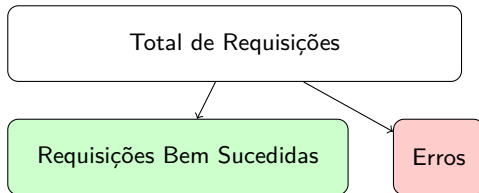


Definição Utilizada pelo Google

Em sistemas distribuídos globais, a disponibilidade é definida como:

$$Disponibilidade = \frac{\text{Requisições bem sucedidas}}{\text{Total de requisições}}$$

A métrica é calculada em uma janela móvel (ex: 1 dia, 1 semana ou 1 trimestre).



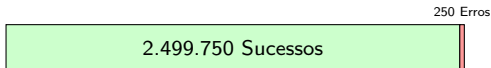
Cenário

- 2.500.000 requisições em um dia
- Meta de disponibilidade: 99,99%

Erro Máximo Permitido

$$0,01\% \times 2.500.000 = 250 \text{ erros}$$

O sistema pode falhar até 250 requisições e ainda atingir a meta diária.



Por Que Medir por Requisições?

Perspectiva do Usuário

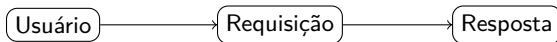
Disponibilidade, na prática, significa:

"Minha requisição funcionou ou não?"

O usuário não percebe horas de uptime. Ele percebe sucesso ou falha da ação executada.

Vantagens do Modelo

- Reflete experiência real do usuário
- Funciona em sistemas parcialmente operacionais
- É diretamente mensurável por engenharia
- Permite metas quantitativas claras



Sucesso ou Falha

Status 100 – Continue (Faixa 1xx – Informacional)

O que significa a faixa 1xx?

A faixa 1xx representa respostas informativas. O servidor recebeu a requisição e está processando, mas ainda não concluiu a resposta final.

Sobre o 100 Continue

O status 100 indica que o cliente pode continuar enviando o corpo da requisição. É comum em requisições POST grandes.

Exemplo comum: envio de relatório grande

Request

```
POST /caldeira/relatorio HTTP/1.1
Host: api.siderurgia.com
Expect: 100-continue
Content-Length: 500000
```

Resposta intermediária

```
HTTP/1.1 100 Continue
```



Interpretação do QA

Status 200 – OK (Faixa 2xx – Sucesso)

O que significa a faixa 2xx?

Indica que a requisição foi recebida, processada e concluída com sucesso.

Sobre o 200 OK

É o código mais comum. Significa que a operação foi executada corretamente.

Exemplo frequente: consulta de status

Request

```
GET /caldeira/status HTTP/1.1  
Host: api.siderurgia.com
```

Response

```
HTTP/1.1 200 OK  
Content-Type: application/json
```

```
{  
  "temperatura": 850,  
  "pressao": 120,  
  "status": "OPERANDO_NORMAL"
```

Status 300 – Redirecionamento (Faixa 3xx)

O que significa a faixa 3xx?

Indica que o recurso solicitado está disponível em outro local ou exige ação adicional do cliente.

Exemplo: 301 Moved Permanently

O endpoint foi alterado permanentemente.

Request

```
GET /caldeira/dados HTTP/1.1  
Host: api.siderurgia.com
```

Response

```
HTTP/1.1 301 Moved Permanently  
Location: /caldeira/status
```

Interpretação do QA

Não é erro. O cliente deve redirecionar automaticamente para o novo endpoint indicado.

Status 404 – Not Found (Faixa 4xx – Erro do Cliente)

O que significa a faixa 4xx?

Erros causados por requisições inválidas, malformadas ou com recurso inexistente. A responsabilidade é do cliente.

Sobre o 404

Indica que o recurso solicitado não existe.

Request

```
GET /caldeira/manutencao/9999 HTTP/1.1
Host: api.siderurgia.com
```

Response

```
HTTP/1.1 404 Not Found
```

```
{
  "erro": "Registro não encontrado"
}
```



Status 404 – Not Found (Faixa 4xx – Erro do Cliente)

Request

```
GET /caldeira/manutencao/9999 HTTP/1.1  
Host: api.siderurgia.com
```

Response

```
HTTP/1.1 404 Not Found
```

```
{  
  "erro": "Registro não encontrado"  
}
```

Interpretação do QA

Erro esperado. Sistema deve responder corretamente sem gerar falha interna.

Status 500 – Internal Server Error (Faixa 5xx)

O que significa a faixa 5xx?

Indica falha interna no servidor. O problema não está na requisição, mas no processamento.

Request

```
POST /caldeira/manutencao HTTP/1.1
Content-Type: application/json

{
  "tecnico": "Carlos Silva"
}
```

Response

```
HTTP/1.1 500 Internal Server Error

{
  "erro": "Falha interna no sistema"
}
```



Interpretação do QA

Status 500 – Internal Server Error (Faixa 5xx)

Request

POST /caldeira/manutencao HTTP/1.1

Content-Type: application/json

```
{  
  "tecnico": "Carlos Silva"  
}
```

Response

HTTP/1.1 500 Internal Server Error

```
{  
  "erro": "Falha interna no sistema"  
}
```

Interpretação do QA

Erro crítico. Impacta disponibilidade. Deve gerar incidente ou alerta.



Estudo de Caso

Teste de uso de sistema

Sistema de Monitoramento de Caldeira Industrial
Siderurgia

Exercício de Teste Manual de QA

GET – Consulta de Status da Caldeira

O que o QA fez

O QA acessou o endpoint de status para verificar se a caldeira está operando dentro dos limites.

Método: GET

Request

```
GET /caldeira/status HTTP/1.1
Host: api.siderurgia.com
Accept: application/json
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "temperatura": 850,
  "pressao": 120,
  "status": "OPERANDO_NORMAL"
}
```

Estudo de Caso

Teste de uso de sistema

Sistema de Monitoramento de Caldeira Industrial
Siderurgia

Exercício de Teste Manual de QA

GET – Consulta de Status da Caldeira

O que o QA fez

O QA acessou o endpoint de status para verificar se a caldeira está operando dentro dos limites.

Método: GET

Request

```
GET /caldeira/status HTTP/1.1
Host: api.siderurgia.com
Accept: application/json
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "temperatura": 850,
  "pressao": 120,
  "status": "OPERANDO_NORMAL"
}
```

POST – Tentativa com Dados Inválidos

O que o QA fez

O QA tentou criar um registro de manutenção com dados incompletos para testar validação.

Método: POST

Request

```
POST /caldeira/manutencao HTTP/1.1
```

```
Content-Type: application/json
```

```
{  
  "tecnico": "",  
  "prioridade": "ALTISSIMA"  
}
```

Response

```
HTTP/1.1 400 Bad Request
```

```
{  
  "erro": "Prioridade inválida"  
}
```

POST – Criação de Manutenção Corretiva

O que o QA fez

Após corrigir os dados, o QA criou corretamente um novo registro de manutenção.

Método: POST

Request

```
POST /caldeira/manutencao HTTP/1.1
```

```
Content-Type: application/json
```

```
{  
  "tecnico": "Ana Souza",  
  "descricao": "Troca de sensor",  
  "prioridade": "ALTA"  
}
```

Response

```
HTTP/1.1 201 Created
```

```
Location: /caldeira/manutencao/52
```



GET – Consulta de Registro Inexistente

O que o QA fez

O QA tentou consultar um registro inexistente para validar tratamento de erro 404.

Método: GET

Request

```
GET /caldeira/manutencao/9999 HTTP/1.1  
Host: api.siderurgia.com
```

Response

```
HTTP/1.1 404 Not Found
```

```
{  
  "erro": "Registro não encontrado"  
}
```



DELETE – Remoção de Registro Criado

O que o QA fez

O QA removeu o registro recém-criado para validar exclusão correta do recurso.

Método: DELETE

Request

```
DELETE /caldeira/manutencao/52 HTTP/1.1  
Host: api.siderurgia.com
```

Response

```
HTTP/1.1 204 No Content
```

DELETE – Falha Interna no Servidor

O que o QA fez

O QA tentou remover um registro durante uma falha simulada no banco de dados.

Método: DELETE

Request

```
DELETE /caldeira/manutencao/60 HTTP/1.1  
Host: api.siderurgia.com
```

Response

```
HTTP/1.1 500 Internal Server Error
```

```
{  
  "erro": "Falha ao acessar banco de dados"  
}
```



Resumo das Execuções do QA

Durante o teste manual foram realizadas 6 requisições utilizando os métodos GET, POST e DELETE.

- GET → 200 OK
- POST → 400 Bad Request
- POST → 201 Created
- GET → 404 Not Found
- DELETE → 204 No Content
- DELETE → 500 Internal Server Error

Totais

- Total de requisições: 6
- Sucessos (2xx): 3
- Erros (4xx e 5xx): 3

Fórmula

$$\text{Disponibilidade} = \frac{\text{Requisições bem sucedidas}}{\text{Total de requisições}}$$

Abordagem 1 – Apenas 2xx como sucesso

$$\text{Disponibilidade} = \frac{3}{6} = 50\%$$

Interpretação

Considerando somente códigos 2xx como sucesso, a disponibilidade observada no teste foi de 50%.

Análise por Classe de Erro

- 4xx → erro causado pelo cliente
- 5xx → erro interno do servidor

Durante o teste houve apenas 1 erro 5xx.

$$Disponibilidade = \frac{5}{6}$$

$$Disponibilidade \approx 83,33\%$$

Conclusão Técnica

Se considerarmos apenas falhas do servidor (5xx), a disponibilidade real do sistema foi de 83,33%.

Erros 4xx indicam que o sistema respondeu corretamente a uma requisição inválida.

